

Python Cơ Bản

Hàm, Module os và Thao tác File

Tóm tắt nội dung

Tài liệu này giới thiệu những kiến thức cơ bản nhất về hàm (function), module os và thao tác với file trong Python. Đây là những kỹ năng nền tảng giúp bạn tự động hóa công việc, xử lý dữ liệu và xây dựng các chương trình thực tế.

Mục lục

1	Giới thiệu về Hàm trong Python	2
1.1	Định nghĩa hàm	2
1.2	Gọi hàm	2
1.3	Tham số và đối số	3
1.4	Giá trị mặc định cho tham số	3
1.5	Hàm không trả về giá trị	3
1.6	Phạm vi biến	4
2	Module os trong Python	4
2.1	Sử dụng module os	4
2.2	Liệt kê file và thư mục	5
2.3	Tạo và xóa thư mục	5
2.4	Đổi tên file/thư mục	5
2.5	Kiểm tra sự tồn tại của file/thư mục	6
3	Thao tác với File	6
3.1	Mở file	6
3.2	Đọc file	6
3.3	Ghi file	6
3.4	Thao tác với file sử dụng with	7
3.5	Đọc file theo dòng	7
3.6	Ghi nhiều dòng vào file	7
4	Bài tập thực hành	7
4.1	Bài tập cơ bản	7
4.2	Bài tập ứng dụng	7
4.3	Bài tập nâng cao	8
5	Kết luận	8

1. Giới thiệu về Hàm trong Python

Hàm (function) là một khối mã được đặt tên, thực hiện một nhiệm vụ cụ thể và có thể tái sử dụng nhiều lần trong chương trình.

Thông tin

Hàm giúp:

- Tránh lặp lại mã nguồn
- Dễ dàng bảo trì và mở rộng chương trình
- Chia nhỏ bài toán phức tạp thành các phần đơn giản

1.1. Định nghĩa hàm

Để định nghĩa một hàm trong Python, ta sử dụng từ khóa `def` theo cú pháp:

```
def ten_ham(tham_so_1, tham_so_2):  
    # Nội dung hàm  
    return gia_tri_tra_ve
```

Ví dụ

Ví dụ: Hàm tính tổng hai số

```
def tinh_tong(a, b):  
    # This function returns the sum of two numbers  
    tong = a + b  
    return tong
```

Giải thích:

- `def` là từ khóa định nghĩa hàm
- `tinh_tong` là tên hàm
- `a, b` là các tham số (parameter)
- `return` trả về kết quả

1.2. Gọi hàm

Sau khi định nghĩa, ta gọi hàm bằng cách sử dụng tên hàm và truyền các đối số (argument) cần thiết.

Ví dụ

Ví dụ: Gọi hàm tính tổng

```
ket_qua = tinh_tong(3, 5)
print(ket_qua) # Output: 8
```

Giải thích:

- `tinh_tong(3, 5)` gọi hàm với hai đối số 3 và 5
- Kết quả trả về được lưu vào biến `ket_qua`
- `print(ket_qua)` in ra màn hình giá trị 8

1.3. Tham số và đối số

- Tham số (parameter) là biến được khai báo trong định nghĩa hàm
- Đối số (argument) là giá trị thực tế truyền vào khi gọi hàm

Ví dụ

Ví dụ: Hàm chào hỏi

```
def chao_hoi(ten):
    # This function greets the user
    print("Xin chao, " + ten + "!")

chao_hoi("Lan") # Output: Xin chao, Lan!
```

Giải thích:

- `ten` là tham số
- `"Lan"` là đối số

1.4. Giá trị mặc định cho tham số

Ta có thể đặt giá trị mặc định cho tham số, nếu không truyền đối số, Python sẽ sử dụng giá trị mặc định.

Ví dụ

Ví dụ: Hàm chào hỏi với tham số mặc định

```
def chao_hoi(ten="Ban"):
    # This function greets the user, default name is "Ban"
    print("Xin chao, " + ten + "!")

chao_hoi() # Output: Xin chao, Ban!
chao_hoi("Nam") # Output: Xin chao, Nam!
```

1.5. Hàm không trả về giá trị

Nếu hàm không có `return`, Python sẽ tự động trả về `None`.

Ví dụ

Ví dụ: Hàm in thông báo

```
def in_thong_bao(thong_diep):  
    # This function prints a message  
    print(thong_diep)  
  
ket_qua = in_thong_bao("Xin chào!")  
print(ket_qua)  # Output: None
```

1.6. Phạm vi biến

- Biến định nghĩa trong hàm chỉ sử dụng được trong hàm đó (biến cục bộ)
- Biến định nghĩa ngoài hàm có thể sử dụng trong toàn bộ chương trình (biến toàn cục)

Ví dụ

Ví dụ: Biến cục bộ và biến toàn cục

```
x = 10  # Global variable  
  
def in_bien():  
    y = 5  # Local variable  
    print("Biến cục bộ y:", y)  
    print("Biến toàn cục x:", x)  
  
in_bien()  
# print(y)  # This will cause an error
```

2. Module os trong Python

Module os cung cấp các hàm để tương tác với hệ điều hành, như thao tác với thư mục và file.

Thông tin

Một số hàm thường dùng trong module os:

- `os.getcwd()`: Lấy đường dẫn thư mục hiện tại
- `os.listdir()`: Liệt kê file và thư mục
- `os.mkdir()`: Tạo thư mục mới
- `os.rename()`: Đổi tên file/thư mục
- `os.remove()`: Xóa file

2.1. Sử dụng module os

Trước khi sử dụng, ta cần import module os.

Ví dụ**Ví dụ: Lấy đường dẫn thư mục hiện tại**

```
import os

duong_dan = os.getcwd()
print("Duong dan hien tai:", duong_dan)
# Output: Duong dan hien tai: /home/user/documents
```

2.2. Liệt kê file và thư mục**Ví dụ****Ví dụ: Liệt kê file và thư mục**

```
import os

danh_sach = os.listdir()
print("Danh sach file va thu muc:", danh_sach)
# Output: Danh sach file va thu muc: ['file1.txt', 'folder1', 'script.py']
```

2.3. Tạo và xóa thư mục**Ví dụ****Ví dụ: Tạo và xóa thư mục**

```
import os

os.mkdir("thu_muc_moi") # Create a new directory
os.rmdir("thu_muc_moi") # Remove the directory
```

2.4. Đổi tên file/thư mục**Ví dụ****Ví dụ: Đổi tên file**

```
import os

os.rename("file_cu.txt", "file_moi.txt")
```

2.5. Kiểm tra sự tồn tại của file/thư mục

Ví dụ

Ví dụ: Kiểm tra file/thư mục có tồn tại không

```
import os

if os.path.exists("file_moi.txt"):
    print("File ton tai!")
else:
    print("File khong ton tai!")
```

3. Thao tác với File

Python cung cấp các hàm để đọc và ghi file, giúp lưu trữ và xử lý dữ liệu lâu dài.

3.1. Mở file

Sử dụng hàm `open()` để mở file, với hai tham số chính:

- Tên file
- Chế độ mở: "r" (đọc), "w" (ghi), "a" (thêm)

Ví dụ

Ví dụ: Mở file để đọc

```
file = open("du_lieu.txt", "r")
```

3.2. Đọc file

Ví dụ

Ví dụ: Đọc toàn bộ nội dung file

```
file = open("du_lieu.txt", "r")
noi_dung = file.read()
print(noi_dung)
file.close()
```

3.3. Ghi file

Ví dụ

Ví dụ: Ghi nội dung vào file

```
file = open("du_lieu.txt", "w")
file.write("Day la noi dung moi.")
file.close()
```

3.4. Thao tác với file sử dụng **with**

Sử dụng **with** giúp tự động đóng file sau khi thao tác xong, tránh lỗi quên đóng file.

Ví dụ

Ví dụ: Sử dụng **with** để đọc file

```
with open("du_lieu.txt", "r") as file:
    noi_dung = file.read()
    print(noi_dung)
# File is automatically closed
```

3.5. Đọc file theo dòng

Ví dụ

Ví dụ: Đọc file theo từng dòng

```
with open("du_lieu.txt", "r") as file:
    for dong in file:
        print(dong, end="")
```

3.6. Ghi nhiều dòng vào file

Ví dụ

Ví dụ: Ghi nhiều dòng vào file

```
dong_1 = "Dong thu nhat\n"
dong_2 = "Dong thu hai\n"

with open("du_lieu.txt", "w") as file:
    file.write(dong_1)
    file.write(dong_2)
```

4. Bài tập thực hành

4.1. Bài tập cơ bản

1. Viết hàm tính tích hai số và gọi hàm với các đối số 4 và 5.
2. Viết hàm in ra bảng cửu chương của một số nguyên dương.
3. Sử dụng module `os` để liệt kê tất cả file trong thư mục hiện tại.
4. Viết chương trình tạo một file mới tên `hello.txt` và ghi vào đó dòng chữ "Xin chào Python!".

4.2. Bài tập ứng dụng

1. Viết hàm tính diện tích hình chữ nhật với hai tham số là chiều dài và chiều rộng.

2. Viết hàm kiểm tra một số có phải là số nguyên tố không.
3. Viết chương trình đọc file `so.txt` chứa các số nguyên, mỗi số một dòng, và in ra tổng của các số đó.
4. Viết chương trình sao chép nội dung từ file `nguồn.txt` sang file `dịch.txt`.

4.3. Bài tập nâng cao

1. Viết hàm tính giai thừa của một số nguyên dương.
2. Viết hàm đếm số lượng file trong một thư mục.
3. Viết chương trình đọc file `ten.txt` chứa danh sách tên học sinh, mỗi tên một dòng, và ghi ra file `ten_moi.txt` với mỗi tên được viết hoa chữ cái đầu.

5. Kết luận

Trong bài học này, bạn đã được giới thiệu về:

- Cách định nghĩa và sử dụng hàm trong Python
- Module `os` và các thao tác với hệ điều hành
- Cách đọc và ghi file trong Python

Mẹo

Mẹo học tập:

- Thực hành viết hàm cho mọi bài toán nhỏ
- Luôn đóng file sau khi thao tác xong
- Sử dụng `with` để quản lý file an toàn
- Đọc tài liệu chính thức của Python để tìm hiểu thêm

Ứng dụng thực tế

Ứng dụng thực tế:

- Tự động hóa công việc hàng ngày với script Python
- Xử lý và phân tích dữ liệu từ file
- Xây dựng các chương trình quản lý file và thư mục